

AWS Solutions Architect Associate — SAA-C03

GUÍA DE ESTUDIO COMPLETA
Con explicaciones, analogías y ejemplos del curso

Basada en el curso de **Stéphane Maarek** y **Johanna Mengual**
Edición en español

Temas cubiertos: Cloud Computing · Infraestructura Global · IAM · EC2 completo
EBS y Almacenamiento · Elastic Load Balancer · Auto Scaling Groups

Certificación AWS · Nivel Associate

Índice

1. ¿Qué es el Cloud Computing?	3
1.1. Los cinco pilares del Cloud Computing (NIST)	3
1.2. Modelos de despliegue	3
1.3. Modelos de servicio	3
1.4. Ventajas del Cloud Computing	3
2. Infraestructura Global de AWS	4
2.1. Regiones	4
2.2. Zonas de Disponibilidad (AZs)	4
2.3. Puntos de Presencia / Edge Locations	4
3. IAM — Gestión de Identidades y Accesos	5
3.1. Usuarios, Grupos y Roles	5
3.2. Políticas IAM — Estructura JSON	5
3.3. Principio de Mínimo Privilegio	6
3.4. MFA — Autenticación Multifactor	6
3.5. Acceso a AWS: Consola, CLI y SDK	7
3.6. Buenas Prácticas de IAM	8
4. EC2 — Elastic Compute Cloud	8
4.1. Tipos de instancias EC2	8
4.2. User Data — Bootstrap al Arranque	9
4.3. Grupos de Seguridad — El Firewall de EC2	9
4.4. Conectarse a EC2 mediante SSH	10
5. EC2 — Opciones de Compra	10
5.1. On-Demand (Bajo Demanda)	11
5.2. Reserved Instances (Instancias Reservadas)	11
5.3. Savings Plans	12
5.4. Spot Instances	12
5.5. Dedicated Hosts (Hosts Dedicados)	13
5.6. Dedicated Instances	13
5.7. Capacity Reservations	13
6. EC2 — Redes y Direcciones IP	13
6.1. IP Pública vs. IP Privada vs. IP Elástica	13
6.2. Grupos de Ubicación (Placement Groups)	14
6.3. ENI — Elastic Network Interface	15
6.4. Hibernación de EC2	15
7. EBS — Elastic Block Store	16
7.1. ¿Qué es EBS?	16
7.2. EBS Snapshots	16
7.3. AMI — Amazon Machine Image	17
7.4. EC2 Instance Store	18
7.5. Tipos de Volúmenes EBS	18
7.5.1. SSD (Solid State Drive) — Baja latencia, alto IOPS	18
7.5.2. HDD (Hard Disk Drive) — Alto throughput, bajo costo	19
7.6. EBS Multi-Attach	19

7.7. Cifrado de EBS	19
8. ELB — Elastic Load Balancer	20
8.1. ¿Para qué sirve un Load Balancer?	20
8.2. Tipos de Load Balancers en AWS	21
8.3. ALB — Application Load Balancer	21
8.4. NLB — Network Load Balancer	22
8.5. GWLB — Gateway Load Balancer	22
8.6. Sticky Sessions (Sesiones Persistentes)	22
8.7. Cross-Zone Load Balancing (Balanceo entre Zonas)	23
8.8. SSL/TLS y SNI	23
8.9. Connection Draining (Drenaje de Conexión)	24
9. ASG — Auto Scaling Groups	24
9.1. ¿Para qué sirven los ASG?	24
9.2. Capacidad del ASG	24
9.3. Launch Template (Plantilla de Lanzamiento)	25
9.4. ASG con Load Balancer	25
9.5. Políticas de Escalado	25
9.6. Métricas y CloudWatch Alarms	26
9.7. Ejemplo Práctico del Curso	26
10. Referencia Rápida para el Examen	27

¿Qué es el Cloud Computing?

Concepto clave

El **Cloud Computing** es la entrega bajo demanda de recursos de computación (servidores, almacenamiento, bases de datos, redes, software) a través de Internet con precios de pago por uso. En lugar de comprar, poseer y mantener centros de datos físicos propios, puedes acceder a recursos tecnológicos, como potencia de cómputo, almacenamiento y bases de datos, según los necesites, desde un proveedor de nube como AWS.

Los cinco pilares del Cloud Computing (NIST)

1. **Autoservicio bajo demanda:** puedes aprovisionar recursos sin intervención humana del proveedor
2. **Amplio acceso a la red:** disponible desde cualquier lugar por Internet
3. **Agrupación de recursos** (multi-tenancy): múltiples clientes comparten la misma infraestructura física
4. **Elasticidad rápida:** escala hacia arriba o abajo según la demanda, rápidamente
5. **Servicio medido:** pagas exactamente lo que usas

Modelos de despliegue

- **Cloud privado:** nube usada por una sola organización, control total, mayor costo (ej. Rackspace)
- **Cloud público:** recursos gestionados por un tercero y entregados por Internet (AWS, Azure, GCP)
- **Cloud híbrido:** mezcla de infraestructura local con cloud público — útil para mantener datos sensibles on-premise y escalar al cloud

Modelos de servicio

- **IaaS** (Infrastructure as a Service): tú gestionas el SO, middleware y apps. AWS te da el hardware virtual. Ejemplo: EC2
- **PaaS** (Platform as a Service): AWS gestiona el SO y plataforma. Tú solo gestionas la app y los datos. Ejemplo: Elastic Beanstalk
- **SaaS** (Software as a Service): producto completo gestionado por el proveedor. Ejemplo: Gmail, Salesforce, Rekognition

Ventajas del Cloud Computing

- Cambias gastos de capital (CAPEX) por gastos operativos (OPEX) — pagas solo lo que consumes
- Te beneficias de enormes economías de escala — AWS tiene millones de clientes, reduce costes
- Dejas de adivinar la capacidad — escala según la demanda real
- Aumentas tu velocidad y agilidad — los recursos están listos en minutos, no semanas

- Dejas de gastar dinero en gestionar centros de datos
- Puedes ser global en minutos — desplegar en múltiples regiones del mundo

Importante para el examen

En el examen: **IaaS** = tú gestionas más (EC2). **PaaS** = AWS gestiona la plataforma (Beanstalk). **SaaS** = todo gestionado (Rekognition, Gmail). Los seis beneficios del cloud son: cambio CAPEX→OPEX, economías de escala, sin adivinar capacidad, velocidad y agilidad, no gestionar CPDs, globalización.

Infraestructura Global de AWS

Regiones

Concepto clave

Una **Región** de AWS es una zona geográfica del mundo donde AWS tiene un cluster de centros de datos. Ejemplos: **us-east-1** (Norte de Virginia), **eu-west-1** (Irlanda), **ap-southeast-1** (Singapur). La mayoría de los servicios de AWS son regionales — están aislados a la región donde los configuras.

¿Cómo elegir la región correcta? Hay cuatro factores clave:

1. **Cumplimiento legal y de datos:** algunos países requieren que los datos no salgan de su territorio (ej. Francia requiere datos en la UE)
2. **Latencia:** elige la región más cercana a tus usuarios para minimizar el tiempo de respuesta
3. **Servicios disponibles:** no todos los servicios de AWS están en todas las regiones; los más nuevos aparecen primero en **us-east-1**
4. **Precios:** varían región a región; Brasil es más caro que EE.UU. por ejemplo

Zonas de Disponibilidad (AZs)

Cada región tiene entre 3 y 6 **Zonas de Disponibilidad** (AZs). Por ejemplo, **us-east-1** tiene: **us-east-1a**, **us-east-1b**, **us-east-1c**, etc.

Cada AZ es uno o más centros de datos físicos con energía, red y conectividad redundantes. Están separadas geográficamente para que un desastre (incendio, inundación, terremoto) no afecte a más de una AZ a la vez. Se conectan entre sí con redes de muy alta velocidad y baja latencia.

Puntos de Presencia / Edge Locations

AWS tiene más de 400 puntos de presencia (PoPs) en más de 90 ciudades del mundo. Se utilizan para **CloudFront** (CDN) y **Route 53** (DNS) para entregar contenido con la menor latencia posible a los usuarios finales. Un edge location no es una región ni una AZ — es solo un punto de caché y entrega de contenido.

Importante para el examen

Datos clave para el examen: AWS tiene **más de 30 regiones** y **más de 100 AZs**. Las AZs están separadas físicamente para resistir desastres. Los Edge Locations son para

CloudFront (entrega de contenido, CDN). El alcance de los servicios: globales (IAM, Route 53, CloudFront), regionales (la mayoría), zonales (subnets, EBS).

IAM — Gestión de Identidades y Accesos

Concepto clave

IAM (Identity and Access Management) es el servicio de AWS que controla **quién** puede acceder a **qué** dentro de tu cuenta de AWS. Es un servicio **global** — no está ligado a ninguna región.

Usuarios, Grupos y Roles

Usuarios IAM: representan a personas reales o servicios. Cada usuario tiene credenciales propias (usuario+contraseña para la consola, Access Key+Secret Key para la API/CLI). Cuando creas una cuenta de AWS, se crea automáticamente el usuario **root** — que tiene acceso total y NO deberías usar en el día a día.

Grupos IAM: son contenedores de usuarios. Un usuario puede pertenecer a varios grupos (o ninguno). Los grupos **no pueden contener otros grupos**, solo usuarios. Cuando asignas una política a un grupo, todos los usuarios del grupo la heredan.

Ejemplo de organización de equipos:

- Grupo *Developers* →políticas de acceso a EC2, S3
- Grupo *Operations* →políticas de acceso a CloudWatch, EC2
- Grupo *Audit* →solo lectura en toda la cuenta
- Alice pertenece a Developers y Audit →tiene todas sus políticas combinadas

Roles IAM: se asignan a **servicios de AWS** (no a personas). Por ejemplo, una instancia EC2 que necesita leer de S3 debe tener un rol con esa política. Los roles son temporales — el servicio asume el rol cuando lo necesita.

Políticas IAM — Estructura JSON

Una política IAM es un documento JSON que define permisos. Estructura completa:

Listing 1: Estructura de una politica IAM

```
{
  "Version": "2012-10-17",           // Version del lenguaje de politica
  "Id": "mi-politica-001",           // Identificador opcional de la politica
  "Statement": [                     // Lista de permisos (puede haber varios)
    {
      "Sid": "AllowEC2Read",          // ID opcional del statement
      "Effect": "Allow",              // Allow o Deny
      "Principal": {                 // Quien aplica (en politicas basadas en
        "AWS": "arn:aws:iam::123456789012:root"
      },
      "Action": [                    // Que acciones
        "ec2:Describe*",
        "ec2:GetConsole*"
      ],
      "Resource": "*",               // Sobre que recursos (ARN o *)
    }
  ]
}
```

```
    "Condition": {                                // Condiciones opcionales
      "StringEquals": {
        "aws:RequestedRegion": "eu-west-1"
      }
    }
  }
}
```

Importante para el examen

Campos del JSON que debes dominar:

- **Version:** siempre "2012-10-17" (la versión actual del lenguaje)
- **Statement:** array de uno o más permisos
- **Effect:** Allow o Deny — el **Deny explícito siempre gana** sobre cualquier Allow
- **Action:** qué operaciones permites/denigas (ej. `s3:GetObject`, `ec2:*`)
- **Resource:** sobre qué recursos aplica (ARN específico o `*` para todos)

Principio de Mínimo Privilegio

Consejo / Buena práctica

Otorga siempre el **mínimo privilegio necesario**. No des a un usuario más permisos de los que necesita para hacer su trabajo. Si alguien solo necesita leer S3, no le des acceso de escritura ni acceso a EC2. Este es el principio de seguridad más importante en IAM.

MFA — Autenticación Multifactor

Concepto clave

MFA (Multi-Factor Authentication) añade una capa extra de seguridad: además de la contraseña, necesitas un segundo factor (código temporal). Si un hacker roba tu contraseña, aún necesitaría tu dispositivo físico para entrar.

Caso concreto del curso: imagina que un hacker logra robarte la contraseña. Sin MFA, tiene acceso total. Con MFA activado, aunque tenga la contraseña, no puede entrar sin el segundo factor.

Dispositivos MFA compatibles con AWS:

- **Virtual MFA:** aplicaciones en el móvil — Google Authenticator (un dispositivo por app), Authy (multi-dispositivo, permite hacer backup). Generan códigos TOTP de 6 dígitos que cambian cada 30 segundos
- **Llave de seguridad física Universal 2nd Factor (U2F):** dispositivo físico USB como **YubiKey** de Yubico. Puedes usar la misma llave para múltiples cuentas de AWS
- **Hardware MFA Token:** dispositivo físico como Gemalto o SurePassID — genera códigos de 6 dígitos que cambian cada 30 segundos. Para casos donde no puedes usar el móvil

Cómo activar MFA en la consola:

1. Ir a la cuenta → Configuración de seguridad

2. Seleccionar “Activar MFA”
3. Elegir el tipo (Virtual MFA Device)
4. Escanear el código QR con Google Authenticator o Authy
5. Introducir dos códigos consecutivos para verificar la sincronización
6. ¡Listo! Ahora cada login pedirá contraseña + código MFA

¡Advertencia!

SIEMPRE activa MFA en la cuenta root. El usuario root tiene acceso total e irreversible a toda la cuenta. Perder el control del root puede ser catastrófico. Recomendación del curso: usa el root solo para crear la cuenta y configurar el primer usuario IAM con permisos de administrador; después **nunca uses el root**.

Acceso a AWS: Consola, CLI y SDK

Hay tres formas de interactuar con AWS:

1. **Consola web:** interfaz gráfica en `console.aws.amazon.com` — requiere usuario+contraseña (+MFA)
2. **AWS CLI** (Command Line Interface): herramienta de terminal que usa Access Keys. Permite automatizar operaciones con scripts. Se instala en tu ordenador.
3. **AWS SDK:** bibliotecas en Python (boto3), Java, JavaScript, etc. Para integrar AWS en aplicaciones

Access Keys: para usar CLI o SDK necesitas un Access Key ID y un Secret Access Key. Son como el usuario y contraseña, pero para acceso programático.

¡Advertencia!

Las Access Keys son secretas — trátalas como contraseñas:

- **NUNCA compartas** tus Access Keys con nadie
- **NUNCA las subas a GitHub** o cualquier repositorio público — hay bots que escanean GitHub buscando keys expuestas y en minutos pueden gastar miles de dólares en tu cuenta
- **NUNCA uses las keys del root** — crea un usuario IAM para eso
- Cada usuario debe tener sus propias keys — si alguien se va de la empresa, solo desactivas sus keys, no las de todos

AWS CloudShell: terminal integrada en la consola de AWS (ícono de terminal en la barra superior). Ya tiene AWS CLI preconfigurada con tus credenciales — no necesitas instalar nada. Disponible en algunas regiones. Los archivos que creas en CloudShell persisten entre sesiones.

Buenas Prácticas de IAM

Consejo / Buena práctica

Resumen de buenas prácticas que el curso recomienda:

1. Nunca usar la cuenta root excepto para la configuración inicial
2. Crear un usuario IAM de administrador y usar ese en el día a día
3. Asignar usuarios a grupos y gestionar permisos a nivel de grupo
4. Crear políticas con el mínimo privilegio necesario
5. Activar MFA para todos los usuarios, especialmente root y administradores
6. Usar roles para dar permisos a servicios de AWS (EC2, Lambda, etc.)
7. Rotar las contraseñas y Access Keys periódicamente
8. Usar el **IAM Credentials Report** y el **IAM Access Advisor** para auditar

Importante para el examen

IAM Credentials Report: informe a nivel de cuenta que lista todos los usuarios y el estado de sus credenciales (contraseña, MFA, Access Keys, fecha de último uso). Útil para auditorías de seguridad.

IAM Access Advisor: muestra qué permisos tiene un usuario y **cuándo se usaron por última vez**. Útil para reducir permisos — si un usuario lleva 6 meses sin usar S3, puedes quitarle ese permiso.

EC2 — Elastic Compute Cloud

Concepto clave

EC2 (Elastic Compute Cloud) es el servicio de servidores virtuales de AWS. Una instancia EC2 es esencialmente un servidor que se ejecuta en la infraestructura de AWS. Es el corazón del cloud: te permite alquilar potencia de computación sin comprar hardware físico.

Tipos de instancias EC2

Las instancias se nombran con un patrón: familia + generación + tamaño. Ejemplo: **m5.xlarge** = familia M (propósito general), generación 5, tamaño xlarge.

Familia	Tipo	Cuándo usarlas
t2/t3	Micro/Small	Desarrollo, pruebas, Free Tier (t2.micro = capa gratuita)
m5/m6	General	Balance CPU/RAM/red. Aplicaciones de nivel medio
c5/c6	Compute Opt.	Alta CPU: servidores web, procesamiento por lotes, HPC
r5/r6	Memory Opt.	Alta RAM: bases de datos en memoria, cachés (Redis, Memcached)
i3	Storage Opt.	Alto I/O de disco (millones de IOPS): bases de datos NoSQL
g4/p3	GPU	Machine Learning, renderizado de vídeo, deep learning

Importante para el examen

t2.micro = capa gratuita de AWS (Free Tier): 750 horas/mes durante 12 meses. Es la instancia que usarás en todas las prácticas del curso. En el examen, cuando veas “Free Tier” piensa en **t2.micro**.

User Data — Bootstrap al Arranque

Cuando lanzas una instancia EC2, puedes proporcionar un script de **User Data**. Este script **se ejecuta una sola vez, automáticamente, en el primer arranque de la instancia**, como usuario root.

Esto se llama **bootstrapping**: arrancar la instancia con comandos automatizados. Sirve para instalar software, actualizar paquetes, configurar servicios, etc.

Ejemplo real del curso — script para instalar Apache y servir una página web:

Listing 2: User Data: instalar Apache en Amazon Linux

```
#!/bin/bash
# Actualizar todos los paquetes instalados
yum update -y
# Instalar servidor web Apache (httpd)
yum install -y httpd
# Iniciar el servicio Apache
systemctl start httpd
# Habilitar Apache para que arranque con el sistema
systemctl enable httpd
# Crear pagina de inicio con la IP privada de la instancia
echo "<h1>Hola Mundo desde $(hostname -f)</h1>" > /var/www/html/index.html
```

Nota / Detalle técnico

El User Data **solo se ejecuta en el primer arranque**. Si paras y reinicias la instancia, el script **NO** se vuelve a ejecutar. Para que se ejecute en cada reinicio, se necesita configuración adicional con cloud-init.

Grupos de Seguridad — El Firewall de EC2

Los **Grupos de Seguridad** (Security Groups) actúan como un firewall virtual alrededor de tus instancias EC2. Controlan el tráfico entrante (inbound) y saliente (outbound).

Funcionamiento:

- Solo contienen reglas de **Allow** (permiso). No hay reglas de deny explícito — lo que no está permitido, está denegado
- Las reglas pueden referenciar IPs o **otros grupos de seguridad**
- Se asocian a instancias, no a la VPC. Puedes asociar múltiples SGs a una instancia
- Son **stateful**: si permites tráfico entrante, la respuesta sale automáticamente

Reglas por defecto:

- Inbound: todo bloqueado (nada entra por defecto)
- Outbound: todo permitido (por defecto, cualquier cosa puede salir)

Puertos que debes conocer para el examen:

- **22** = SSH (Secure Shell) — acceso remoto a instancias Linux
- **21** = FTP (File Transfer Protocol)
- **22** = SFTP (Secure FTP, también usa puerto 22)

- **80** = HTTP (web sin cifrar)
- **443** = HTTPS (web cifrada, SSL/TLS)
- **3389** = RDP (Remote Desktop Protocol) — acceso a Windows

Importante para el examen

Diagnóstico de errores con Security Groups:

- **Timeout** (la conexión nunca responde, rueda girando indefinidamente): el Security Group está bloqueando el tráfico. Regla faltante o IP origen equivocada
- **“Connection refused”** (responde pero rechaza): el Security Group SÍ permite el tráfico, pero la **aplicación** no está funcionando o no está escuchando en ese puerto

Esta distinción es fundamental: timeout = problema de red/SG; connection refused = problema de aplicación.

Conectarse a EC2 mediante SSH

Para conectarte por SSH a una instancia Linux necesitas el archivo de clave privada `.pem` que descargaste al crear el par de claves.

- **Mac/Linux**: usar directamente `.pem` con el cliente SSH nativo
- **Windows**: usar `.ppk` (convertido con PuTTYgen) para PuTTY, o bien `.pem` directamente en PowerShell (Windows 10+) o WSL

```
# Antes: cambiar permisos del archivo de clave (obligatorio)
chmod 0400 mi-clave.pem

# Conectarse por SSH
ssh -i mi-clave.pem ec2-user@<IP-PUBLICA-DE-LA-INSTANCIA>
```

Nota / Detalle técnico

El usuario predeterminado varía según el AMI:

- Amazon Linux: `ec2-user`
- Ubuntu: `ubuntu`
- CentOS: `centos`
- Windows: `Administrator`

EC2 Instance Connect: alternativa al SSH clásico, disponible directamente desde la consola de AWS. Abre una terminal en el navegador sin necesitar claves. Funciona solo con Amazon Linux 2 y Ubuntu.

EC2 — Opciones de Compra

Analogía del curso

Analogía del Hotel Resort (del curso):

Imagina que quieres quedarte en un hotel resort. Tienes varias opciones de cómo pagar tu estancia:

1. **On-Demand** (Bajo Demanda): llegas al hotel sin reserva y preguntas por una habitación. Pagas el precio completo por hora/día. Perfecto si no sabes cuánto tiempo te vas a quedar. Es la opción más cara por unidad de tiempo, pero no hay compromiso.
2. **Reserved** (Reservado): llamas al hotel con 1 o 3 años de antelación y reservas la habitación. A cambio, te hacen un gran descuento — hasta 72 % sobre el precio de On-Demand. Pagas por adelantado (total, parcial o al final) y obtienes más descuento cuanto más pagas por adelantado.
3. **Savings Plans**: en lugar de reservar un tipo específico de habitación, haces un compromiso de gasto. Le dices al hotel: “voy a gastar mínimo 10 dólares por hora durante 1 o 3 años”. Con eso, obtienes descuento sobre cualquier tipo de habitación dentro de ese gasto.
4. **Spot Instances**: es como las habitaciones de último minuto que el hotel ofrece a precio muy reducido porque están vacías. PERO: si alguien llega y puja más que tú, te sacan de la habitación con 2 minutos de aviso. Es el precio más bajo (hasta 90 % de descuento) pero con riesgo de interrupción.
5. **Dedicated Host** (Host Dedicado): alquilas **todo el edificio del hotel**. El hardware físico es solo tuyo. Necesario por regulaciones de cumplimiento (compliance) o para usar licencias de software ligadas al hardware.
6. **Capacity Reservations**: reservas la habitación aunque no la vayas a usar. El hotel te la guarda y pagas aunque no te quedes. Sin compromisos de tiempo — puedes cancelar cuando quieras, pero siempre pagas.

On-Demand (Bajo Demanda)

- Pagas por segundos (Linux/Windows) o por hora (otros SO)
- El precio más alto pero **sin compromisos**
- Sin contrato a largo plazo — puedes terminar cuando quieras
- **Cuándo usar**: aplicaciones con carga impredecible, desarrollo y pruebas, aplicaciones de corta duración

Reserved Instances (Instancias Reservadas)

- Hasta **72 % de descuento** sobre On-Demand
- Reservas por **1 año o 3 años** (3 años = mayor descuento)
- Pagas: todo por adelantado (mayor descuento), parcialmente, o sin pago inicial (menor descuento)
- Debes especificar el tipo de instancia (ej. **m5.xlarge**), SO y región
- **Convertible Reserved Instances**: permiten cambiar el tipo de instancia, SO y tenencia durante el periodo. Menos descuento (66 %) pero más flexibilidad
- **Cuándo usar**: aplicaciones con uso estable y predecible a largo plazo (bases de datos, servidores web de producción)
- Puedes vender tus reservas en el **Reserved Instance Marketplace** si ya no las necesitas

Savings Plans

- Compromiso de gasto en dólares/hora durante **1 o 3 años** (ej. \$10/hora)
- Hasta **72 % de descuento** como las Reserved Instances
- **Compute Savings Plan**: más flexible — aplica a cualquier familia de instancias, región, SO, tenencia e incluso Lambda y Fargate
- **EC2 Instance Savings Plan**: específico para una familia en una región, más descuento
- Lo que gastes más allá del compromiso se factura a precio On-Demand

Spot Instances

Concepto clave

Las **Spot Instances** te permiten pujar por capacidad no utilizada de AWS. El precio varía dinámicamente según la oferta y demanda. Puedes obtener descuentos de hasta **90 %** comparado con On-Demand. Sin embargo: si alguien ofrece más que tú (o AWS necesita esa capacidad), tu instancia puede ser **interrumpida con solo 2 minutos de aviso**.

Cómo funciona el precio Spot:

- Define un **precio máximo que estás dispuesto a pagar** (max price)
- AWS tiene un **precio Spot actual** que fluctúa según disponibilidad
- Mientras tu max price > precio Spot actual: tu instancia corre
- Si el precio Spot sube por encima de tu max price: te dan 2 minutos de aviso y la instancia se termina (o se detiene, según la configuración)

Cuándo usar Spot Instances:

- Trabajos por lotes (batch processing)
- Análisis de datos, análisis de imágenes
- Cargas de trabajo distribuidas y tolerantes a fallos
- Entornos de desarrollo y pruebas con horario flexible

¡Advertencia!

NUNCA usar Spot Instances para:

- Bases de datos de producción críticas
- Aplicaciones que no pueden interrumpirse
- Servidores web que deben estar siempre disponibles sin interrupciones

Importante para el examen

Proceso correcto para cancelar Spot Instances (pregunta de examen):

Si quieres terminar instancias Spot definitivamente, el orden importa:

1. Primero: **cancela la Spot Request** (si la dejas activa, AWS relanzará nuevas

instancias automáticamente)

2. Después: **termina las instancias Spot** que ya están corriendo

Si terminas las instancias sin cancelar la request primero, AWS lanzará nuevas instancias para cumplir la request.

Spot Fleets: colección de Spot Instances (y opcionalmente On-Demand) que intenta cumplir la capacidad que pides al menor coste posible. Define múltiples tipos de instancias como “pools” y el Spot Fleet elige dónde lanzar para maximizar el ahorro o la disponibilidad.

Dedicated Hosts (Hosts Dedicados)

- Un **servidor físico dedicado exclusivamente a ti**
- El más caro de todos los tipos
- **Cuándo usar:** cumplimiento regulatorio que requiere hardware dedicado, software con licencias ligadas a “sockets” o “cores” físicos (ej. Windows Server, SQL Server, Oracle)
- Puedes reservarlo por 1 o 3 años

Dedicated Instances

- Las instancias corren en hardware dedicado a tu cuenta pero puede compartir el mismo host con otras instancias tuyas
- Sin control sobre la colocación de las instancias (diferencia clave con Dedicated Host)
- Menos control que Dedicated Host, pero también menos caro

Capacity Reservations

- Reservas capacidad de EC2 en una AZ específica por cualquier duración
- Sin compromiso de tiempo — creas y cancelas cuando quieras
- Siempre pagas el precio On-Demand aunque no uses las instancias
- Combínalas con Reserved Instances o Savings Plans para obtener descuentos
- **Cuándo usar:** cuando necesitas garantizar capacidad disponible en una AZ para eventos predecibles de corta duración

Tipo	Descuento	Ideal para
On-Demand	0 % (precio base)	Corto plazo, impredecible
Reserved (1 año)	hasta 40 %	Largo plazo, estable
Reserved (3 años)	hasta 72 %	Muy largo plazo
Savings Plans	hasta 72 %	Flexibilidad con compromiso
Spot	hasta 90 %	Tolerante a interrupciones
Dedicated Host	Sin descuento (más caro)	Compliance, licencias

EC2 — Redes y Direcciones IP

IP Pública vs. IP Privada vs. IP Elástica

IP Privada: identifica la instancia dentro de la red interna de AWS (VPC). Solo accesible desde dentro de la misma red privada. La IP privada **no cambia** cuando paras y reinicias la

instancia.

IP Pública: identifica la instancia en Internet. Debe ser única a nivel mundial. Te permite acceder a la instancia desde Internet (via SSH, HTTP, etc.). **Importante del curso:** la IP pública **cambia automáticamente** cada vez que paras y reinicias la instancia.

Nota / Detalle técnico

Demostración práctica del curso: si paras una instancia EC2 y la reinicias, la dirección IP pública cambiará. La IP privada permanece igual. Esto se observó directamente en la consola: antes del reinicio era 18.206.94.14, después del reinicio fue una IP diferente.

IP Elástica (Elastic IP): dirección IP pública **fija** que pertenece a tu cuenta hasta que la liberes explícitamente. Te permite tener siempre la misma IP pública aunque pares y reinicies la instancia.

Características:

- Solo puedes tener **5 Elastic IPs** por cuenta (puedes pedir aumento a AWS)
- Se puede asociar solo a **una instancia** a la vez
- Si la Elastic IP no está asociada a ninguna instancia en ejecución, **se te cobra** (para evitar el desperdicio de IPs públicas)

Consejo / Buena práctica

Recomendación del curso: evita usar Elastic IPs. Suelen reflejar malas decisiones de arquitectura. Alternativas mejores:

- Usar un **DNS** con nombre de dominio apuntando a la IP pública aleatoria
- Usar un **Load Balancer** con nombre DNS estático (mejor práctica)

Grupos de Ubicación (Placement Groups)

Los Placement Groups controlan cómo AWS coloca físicamente tus instancias EC2 en el hardware. Hay tres estrategias:

1. Cluster — todas las instancias en el **mismo rack**, misma AZ:

- Latencia ultra baja entre instancias (red de 10 Gbps entre ellas)
- Gran rendimiento de red
- Riesgo: si el rack falla, todas las instancias caen a la vez
- **Cuándo usar:** HPC (High Performance Computing), Big Data que necesita completarse rápido, aplicaciones de latencia extremadamente baja

2. Spread (Distribuido) — cada instancia en **hardware diferente**:

- Puede abarcar múltiples AZs
- Máximo 7 instancias por AZ por grupo
- Riesgo minimizado: si un hardware falla, solo cae esa instancia

- **Cuándo usar:** aplicaciones críticas donde cada instancia debe estar aislada de los fallos de las demás

3. Partition (Partición) — grupos de instancias en distintos racks:

- Hasta 7 particiones por AZ, cientos de instancias en total
- Instancias en la misma partición comparten rack, pero particiones diferentes tienen racks diferentes
- **Cuándo usar:** sistemas distribuidos como Kafka, Cassandra, HDFS (Hadoop)

ENI — Elastic Network Interface

Una **ENI** es una tarjeta de red virtual que representa la interfaz de red de una instancia EC2. Sus atributos:

- Una o más **direcciones IP privadas**
- Una **IP pública**
- Una o más **IP elásticas**
- Uno o más **grupos de seguridad**
- Una **dirección MAC**

Las ENIs se pueden crear de forma independiente y **mover en caliente** (mientras las instancias están corriendo) entre instancias de la misma AZ. Útil para conmutación por error (*failover*): si una instancia falla, mueves su ENI a otra instancia y la comunicación continúa con la misma IP privada.

Hibernación de EC2

La **hibernación** es diferente a parar o terminar una instancia:

- **Parar (Stop):** el SO se apaga, los datos del EBS se mantienen. Arrancar de nuevo requiere reiniciar todo el SO
- **Terminar (Terminate):** la instancia se destruye (y el EBS raíz por defecto)
- **Hibernar:** el estado de la RAM se guarda en el volumen EBS raíz (cifrado). Al reiniciar, la RAM se restaura y la instancia arranca donde la dejaste, mucho más rápido

Requisitos de la hibernación:

- El volumen EBS raíz debe estar **cifrado**
- RAM máxima: 150 GB
- Familias soportadas: C, M, R, I, etc. (no instancias *bare metal*)
- Una instancia no puede estar hibernada más de **60 días**

Importante para el examen

Casos de uso de la hibernación: procesamiento de larga duración que quieres interrumpir y continuar; servicios que tardan mucho en inicializarse (guardar la RAM evita el costoso proceso de calentamiento).

EBS — Elastic Block Store

¿Qué es EBS?

Concepto clave

Un **volumen EBS** (Elastic Block Store) es una **unidad de red** (no física) que se puede adjuntar a instancias EC2. Es como una memoria USB de red: la instancia lee y escribe datos a través de la red hacia el volumen EBS.

Analogía del curso: piensa en EBS como un **pendrive de red** que puedes enchufar y desenchufar de tus instancias.

Características clave:

- Es una **unidad de red** — usa la red de AWS para comunicarse con la instancia (puede haber algo de latencia)
- **Persistente:** los datos se mantienen aunque la instancia se pare o termine
- **Ligado a una AZ:** un volumen EBS creado en `us-east-1a` no puede adjuntarse a una instancia en `us-east-1b`. Para moverlo entre AZs se usa un Snapshot
- Normalmente adjunto a **una sola instancia** a la vez (excepto EBS Multi-Attach para IO1/IO2)
- Facturado por la **capacidad provisionada** (en GB) aunque no uses todo el espacio
- **Free Tier:** 30 GB de EBS tipo SSD (GP2 o magnético) gratis por mes

Comportamiento al terminar instancia (Delete on Termination):

- El volumen **raíz (root)**: por defecto **SÍ se elimina** cuando la instancia termina
- Volúmenes adicionales: por defecto **NO se eliminan** cuando la instancia termina
- Puedes cambiar este comportamiento al lanzar la instancia en “Configuración de almacenamiento”

EBS Snapshots

Un **snapshot** (instantánea) es una **copia de seguridad** de un volumen EBS en un momento dado.

- No es necesario desconectar el volumen para hacer el snapshot, aunque se recomienda para asegurar consistencia
- Los snapshots se guardan en S3 (internamente gestionado por AWS)
- Se pueden **copiar entre regiones y AZs** — así puedes mover un volumen EBS de una AZ a otra

- Los snapshots son **incrementales**: solo guardan los bloques que cambiaron desde el último snapshot

Cómo mover un EBS de una AZ a otra:

1. Hacer un snapshot del volumen en AZ origen
2. Restaurar el snapshot como un nuevo volumen en la AZ destino
3. Adjuntar el nuevo volumen a la instancia en la AZ destino

Características adicionales de Snapshots:

- **EBS Snapshot Archive**: mover el snapshot a un nivel de archivo — **75 % más barato**. La restauración tarda entre 24 y 72 horas
- **Recycle Bin**: papelera de reciclaje para snapshots eliminados accidentalmente. Retención configurable de **1 día a 1 año**. Si borras un snapshot por error, lo puedes recuperar si está en el periodo de retención

AMI — Amazon Machine Image

Concepto clave

Una **AMI** (Amazon Machine Image) es como una **plantilla de instancia personalizada**. Incluye el sistema operativo, configuraciones, software preinstalado y configuraciones. Permite lanzar instancias ya configuradas — más rápido que configurar desde cero.

Tipos de AMI:

- **AMI pública de AWS**: Amazon Linux 2, Ubuntu, Windows, etc. Proporcionadas y mantenidas por AWS
- **Tu propia AMI**: la creas tú a partir de una instancia configurada
- **AWS Marketplace AMI**: AMIs vendidas por terceros, que pueden incluir software comercial ya configurado (ej. un servidor Nginx preconfigurado y optimizado)

Proceso de creación de AMI:

1. Lanzas una instancia EC2 y la configuras como quieres (instalar software, configurar servicios)
2. Paras la instancia (para integridad de datos)
3. Haces clic derecho → “Crear imagen” (AMI)
4. AWS crea la AMI (que internamente crea snapshots de EBS)
5. Ahora puedes lanzar nuevas instancias desde esta AMI con todo ya preconfigurado

Las AMIs son regionales — se crean en una región pero puedes copiarlas entre regiones.

EC2 Instance Store

Concepto clave

El **Instance Store** es el almacenamiento físico del hardware donde corre la instancia. Es el disco duro físico del servidor, no un volumen de red como EBS.

Ventajas: rendimiento extremadamente alto:

- EBS: hasta 32.000 IOPS (operaciones de I/O por segundo)
- Instance Store (familia I3): **3.3 millones de IOPS** — más de 100 veces más rápido

Desventajas — EFÍMERO:

- Si la instancia se para o termina, **los datos se pierden para siempre**
- Si el hardware físico falla, **los datos se pierden**
- La copia de seguridad y replicación es **responsabilidad tuya**

Importante para el examen

En el examen, “alto rendimiento de disco” o “millones de IOPS” → piensa en **EC2 Instance Store**. Casos de uso: caché temporal, buffer, datos de scratch, contenido temporal que se puede regenerar.

Tipos de Volúmenes EBS

Hay 6 tipos de volúmenes EBS divididos en dos familias:

SSD (Solid State Drive) — Baja latencia, alto IOPS

GP2 y GP3 — SSD de Propósito General:

- Balance entre precio y rendimiento
- Pueden ser volúmenes de arranque (boot volume)
- Rango: 1 GB a 16 TB
- **GP3:** línea base de 3.000 IOPS y 125 MB/s. Se puede escalar de forma **independiente** hasta 16.000 IOPS y 1.000 MB/s. Más moderno y recomendado
- **GP2:** IOPS vinculadas al tamaño del volumen (3 IOPS/GB). Máximo 16.000 IOPS con volúmenes de 5.334 GB. Los volúmenes pequeños pueden “reventar” hasta 3.000 IOPS

IO1 e IO2 — SSD de alto rendimiento (IOPS Provisionadas):

- Para aplicaciones críticas que necesitan más de 16.000 IOPS
- Pueden ser volúmenes de arranque
- Las IOPS son completamente **independientes del tamaño** del volumen
- **IO1:** 4 GB a 16 TB. Hasta 64.000 IOPS (instancias Nitro) o 32.000 (otras). Casos de uso: bases de datos críticas

- **IO2**: igual que IO1 pero con más durabilidad y más IOPS/GB al mismo precio
- **IO2 Block Express**: 4 GB a 64 TB. Hasta **256.000 IOPS**. Latencia menor de 1 ms. Soporta **EBS Multi-Attach**

HDD (Hard Disk Drive) — Alto throughput, bajo costo

ST1 — HDD de Rendimiento Optimizado:

- **No puede ser volumen de arranque**
- Diseñado para acceso frecuente y alto throughput (no alto IOPS)
- Casos de uso: Big Data, almacenes de datos, procesamiento de logs
- Rendimiento: hasta 500 MB/s de throughput, 500 IOPS
- Rango: 125 GB a 16 TB

SC1 — HDD Frío (Cold):

- **No puede ser volumen de arranque**
- El **más barato** de todos los tipos EBS
- Para datos a los que se accede con poca frecuencia
- Rendimiento: hasta 250 MB/s de throughput, 250 IOPS
- Rango: 125 GB a 16 TB

Tipo	Familia	Boot	Cuándo usar
GP3	SSD	Sí	Propósito general, recomendado por defecto
GP2	SSD	Sí	Propósito general (versión anterior de GP3)
IO1	SSD	Sí	BD críticas, >16.000 IOPS necesarias
IO2	SSD	Sí	Igual IO1 + más durabilidad y IOPS
IO2 Block Express	SSD	Sí	Máximo rendimiento (256K IOPS)
ST1	HDD	No	Big Data, logs, acceso frecuente, alto throughput
SC1	HDD	No	Archivo, acceso infrecuente, menor costo

EBS Multi-Attach

Normalmente un volumen EBS solo puede adjuntarse a una instancia. Con **Multi-Attach** (disponible solo para IO1/IO2), el mismo volumen puede adjuntarse a **hasta 16 instancias EC2 en la misma AZ**. Cada instancia tiene permisos completos de lectura/escritura.

Casos de uso: mayor disponibilidad en clusters Linux, aplicaciones que necesitan escrituras concurrentes.

Cifrado de EBS

Cuando creas un volumen EBS cifrado:

- Los datos en reposo están cifrados dentro del volumen
- Los datos en tránsito entre la instancia y el volumen están cifrados

- Todos los snapshots derivados están cifrados
- Todos los volúmenes creados desde snapshots cifrados también están cifrados
- El cifrado usa **AWS KMS** con algoritmo AES-256
- Impacto en latencia: mínimo

Cómo cifrar un volumen no cifrado existente:

1. Crear un snapshot del volumen no cifrado
2. Copiar el snapshot activando el cifrado durante la copia
3. Crear un nuevo volumen desde el snapshot cifrado
4. Reemplazar el volumen original con el nuevo cifrado

ELB — Elastic Load Balancer

¿Para qué sirve un Load Balancer?

Concepto clave

Un **Load Balancer** (Balanceador de Carga) distribuye el tráfico entrante entre múltiples instancias EC2. Actúa como el único punto de contacto para los clientes — ellos solo conocen la dirección del load balancer, no la de las instancias individuales.

Analogía del curso

Analogía del curso — Escalabilidad:

Imagina que tienes una centralita telefónica de atención al cliente:

- **Escalado vertical** (scale up/down): cambias al operador junior por uno senior con más experiencia y capacidad. Tiene un límite — no puedes crear al operador perfecto infinitamente capaz. En EC2: pasar de **t2.micro** a **c5.4xlarge**
- **Escalado horizontal** (scale out/in): en lugar de un superoperador, pones 5, 6 o 7 operadores trabajando a la vez. Sin límite teórico. En EC2: añadir más instancias del mismo tipo
- **Alta Disponibilidad**: tienes operadores en el edificio de Nueva York Y operadores en el edificio de San Francisco. Si el edificio de Nueva York se quema, los de San Francisco siguen atendiendo. En EC2: instancias en múltiples AZs

Beneficios del Load Balancer:

- Distribuye la carga entre múltiples instancias
- Expone un único punto de entrada (DNS) a la aplicación
- Realiza comprobaciones de salud (**health checks**) a las instancias — deja de enviar tráfico a instancias no saludables
- Proporciona terminación SSL/TLS (gestiona HTTPS)
- Alta disponibilidad entre zonas (Cross-AZ)
- Separa el tráfico público del tráfico privado

Health Checks: cada pocos segundos, el load balancer envía una petición a las instancias (típicamente `GET /health` en el puerto configurado). Si la instancia responde con HTTP 200 (OK), está sana. Si no responde o responde con error, se marca como no saludable y el LB deja de enviarle tráfico.

Tipos de Load Balancers en AWS

AWS ofrece cuatro tipos de load balancers (tres de nueva generación):

Tipo	Capa	Protocolos	Cuándo usar
ALB	7 (App)	HTTP, HTTPS, WebSocket	Tráfico web, routing avanzado
NLB	4 (Trans.)	TCP, UDP, TLS	Ultra-rendimiento, millones de req/s
GWLB	3 (Red)	IP (GENEVE)	Inspección de tráfico, firewalls
Classic LB	4+7	HTTP, HTTPS, TCP	<i>Obsoleto, ya eliminado del dashboard</i>

ALB — Application Load Balancer

El **ALB** opera en la Capa 7 (capa de aplicación). Entiende el contenido de las peticiones HTTP/HTTPS.

Target Groups (Grupos Objetivo): el ALB envía el tráfico a “bolsas de instancias” organizadas en grupos. Como explicó el curso: un target group es “una bolsa llena de instancias que operan como equipo”. Los targets pueden ser:

- Instancias EC2 (individualmente o gestionadas por Auto Scaling Group)
- Contenedores ECS
- Funciones Lambda
- Direcciones IP privadas

Routing avanzado — El ALB puede enrutar según:

- **Ruta URL:** `/users` → Target Group 1, `/search` → Target Group 2
- **Hostname:** `one.example.com` → Target Group 1, `other.example.com` → Target Group 2
- **Parámetros de query string:** `?platform=mobile` → Target Group móvil, `?platform=desktop` → Target Group desktop
- **Headers HTTP:** por cualquier cabecera de la petición

Nota / Detalle técnico

IP real del cliente con ALB: El ALB termina la conexión del cliente y establece una nueva conexión hacia las instancias. Por esto, la instancia EC2 recibe como IP origen la IP del ALB, no la del cliente real. Para obtener la IP real del cliente, el ALB añade la cabecera **X-Forwarded-For** con la IP original del cliente. También añade **X-Forwarded-Port** (puerto original) y **X-Forwarded-Proto** (protocolo HTTP/HTTPS).

Importante para el examen

El ALB es ideal para:

- Microservicios (cada servicio con su target group)
- Aplicaciones basadas en contenedores (ECS)
- Enrutamiento por URL, hostname o parámetros (routing avanzado)
- Cualquier escenario que mencione HTTP/HTTPS en el examen

Seguridad con ALB: puedes configurar el Security Group de tus instancias EC2 para que solo acepten tráfico del propio ALB (usando el SG del ALB como origen). Así los usuarios no pueden acceder directamente a las instancias, solo a través del load balancer.

NLB — Network Load Balancer

El **NLB** opera en la **Capa 4** (transporte). No entiende el contenido HTTP — solo reenvía paquetes TCP/UDP.

Características clave:

- Maneja **millones de peticiones por segundo**
- Latencia ultra baja: **100 ms** vs los 400 ms del ALB
- Tiene una **IP estática por AZ** (puede ser una Elastic IP) — útil para poner en lista blanca una IP fija
- Los health checks soportan protocolos TCP, HTTP y HTTPS
- **No está incluido en la capa gratuita de AWS** (Free Tier)

Importante para el examen

Claves para el examen: si ves **UDP, TCP, millones de peticiones/segundo, baja latencia extrema, IP estática para whitelisting** →piensa en **NLB**. Si ves **HTTP, HTTPS, routing avanzado** →piensa en **ALB**.

GWLB — Gateway Load Balancer

El **GWLB** opera en la **Capa 3** (red). Se usa para inspeccionar el tráfico con **dispositivos virtuales de terceros** (firewalls, sistemas de detección de intrusiones, inspección de paquetes).

Flujo típico: Usuario →GWLB →Target Group (appliances de seguridad de terceros) →GWLB →Tu aplicación.

Usa el protocolo **GENEVE** en el puerto **6081**.

Importante para el examen

Si el examen menciona **GENEVE, puerto 6081, inspección de tráfico, firewall virtual, IDS/IPS, appliances de red de terceros** →piensa en **GWLB**.

Sticky Sessions (Sesiones Persistentes)

Las **Sticky Sessions** permiten que el mismo cliente siempre sea redirigido a la misma instancia tras el load balancer. Se implementa mediante una **cookie**.

Caso de uso: mantener los datos de sesión del usuario en la misma instancia (ej. carrito de

compras almacenado en la instancia, no en base de datos compartida).

Tipos de cookies:

- **Cookie generada por el LB** (Application-Based - LB generated): nombre AWSALB para ALB. El LB la crea y gestiona. Duración configurable: 1 segundo a 7 días
- **Cookie generada por la aplicación:** el nombre lo defines tú, la crea el target. Permite atributos personalizados

¡Advertencia!

Las Sticky Sessions pueden **desequilibrar la carga**: si algunos usuarios hacen más peticiones que otros y siempre van a la misma instancia, esa instancia puede sobrecargarse mientras otras están inactivas. Úsalas solo cuando sea necesario.

Nombres de cookies **reservados por AWS** (no puedes usar estos nombres en tus cookies de aplicación): AWSALB, AWSALBAPP, AWSALBTG

Cross-Zone Load Balancing (Balanceo entre Zonas)

Con Cross-Zone activado: cada nodo del load balancer distribuye el tráfico de forma uniforme entre **todas** las instancias registradas en **todas las AZs**. Si el LB recibe 50 % del tráfico en AZ1 y tiene 2 instancias en AZ1 y 8 instancias en AZ2, **cada instancia recibe 10 %**.

Sin Cross-Zone: cada nodo del LB solo distribuye el tráfico a las instancias en su propia AZ. El 50 % del tráfico que llega a AZ1 se reparte solo entre las 2 instancias de AZ1 (25 % cada una); el 50 % restante se reparte entre las 8 instancias de AZ2 (6.25 % cada una).

Tipo de LB	Cross-Zone por defecto	Coste extra
ALB	Siempre activado (no se puede desactivar)	Sin costo adicional
NLB y GWLB	Desactivado por defecto	Se cobra si se activa

SSL/TLS y SNI

Un **certificado SSL/TLS** cifra el tráfico en tránsito entre el cliente y el load balancer (HTTPS). El LB termina la conexión SSL y establece una conexión no cifrada (HTTP) hacia las instancias en la red privada.

Gestión de certificados: AWS Certificate Manager (**ACM**) permite crear y gestionar certificados SSL gratuitamente. También puedes importar tus propios certificados.

SNI — Server Name Indication: protocolo que permite que un load balancer sirva **múltiples certificados SSL** para **múltiples dominios**. El cliente indica en el handshake inicial el hostname al que quiere conectarse, y el LB devuelve el certificado correspondiente.

Importante para el examen

SNI clave para el examen:

- SNI resuelve el problema de múltiples certificados en un servidor
- **Funciona con:** ALB (generación 2), NLB (generación 2), CloudFront
- **NO funciona con:** Classic Load Balancer (generación 1, obsoleto)
- Si el examen menciona múltiples dominios con distintos certificados SSL en un LB → SNI en ALB o NLB

Connection Draining (Drenaje de Conexión)

El **Connection Draining** (en ALB/NLB se llama **Deregistration Delay**) es el tiempo que se da a una instancia que va a desconectarse para **completar las peticiones en vuelo** antes de eliminarse del target group.

Durante el drenaje:

- No se envían **nuevas** peticiones a esa instancia
- Las peticiones existentes (en vuelo) tienen tiempo de completarse
- Duración: **1 a 3600 segundos** (por defecto: **300 segundos**)
- Poniendo a 0 se desactiva (se desconecta inmediatamente, cortando peticiones en curso)

Consejo / Buena práctica

Recomendación: si tus peticiones son cortas (milisegundos), establece un valor bajo (ej. 30 segundos). Si son largas (uploads de archivos grandes, transacciones complejas), usa un valor alto. El valor de 300 segundos por defecto está bien para la mayoría de casos.

ASG — Auto Scaling Groups

¿Para qué sirven los ASG?

Concepto clave

Un **Auto Scaling Group** (ASG) es un grupo de instancias EC2 que se escala automáticamente según la demanda. AWS añade o quita instancias del grupo para mantener el nivel de rendimiento deseado y el costo optimizado.

La carga de una aplicación puede variar enormemente. El curso usa el ejemplo de Netflix: tiene muchos más usuarios el fin de semana que entre semana. Con un ASG, Netflix no necesita mantener capacidad para el pico del fin de semana todo el tiempo — AWS la escala automáticamente cuando la necesita.

Objetivos de un ASG:

- **Scale out:** añadir instancias cuando la carga aumenta
- **Scale in:** eliminar instancias cuando la carga disminuye (ahorra costo)
- Asegurar un número mínimo y máximo de instancias en todo momento
- Registrar automáticamente nuevas instancias en el load balancer
- Reemplazar instancias no saludables automáticamente
- **Los ASG son gratuitos** — solo pagas por las instancias EC2 que crea

Capacidad del ASG

- **Mínimo (Minimum size):** número mínimo de instancias que siempre deben estar activas. El ASG nunca baja de este número

- **Deseado (Desired capacity)**: el número de instancias que quieres tener normalmente. El ASG intenta mantener este número
- **Máximo (Maximum size)**: el límite superior — el ASG nunca crea más instancias que este número aunque la carga sea enorme

Ejemplo: tienes una tienda online. Configuración del ASG:

- Mínimo: 2 instancias (siempre activas para disponibilidad)
- Deseado: 4 instancias (carga normal del día a día)
- Máximo: 10 instancias (para picos de Black Friday)

Launch Template (Plantilla de Lanzamiento)

Un ASG necesita saber qué tipo de instancias crear. Esto se define en una **Launch Template** (antes llamada Launch Configuration, ya obsoleta). La plantilla incluye:

- AMI (imagen base)
- Tipo de instancia (ej. `t2.micro`)
- Par de claves SSH
- Grupos de seguridad
- Datos de usuario (User Data)
- Volúmenes EBS
- Roles IAM
- Información de red y subredes

ASG con Load Balancer

El ASG trabaja en conjunto con el load balancer:

- Cuando el ASG lanza una nueva instancia, la registra automáticamente en el target group del LB
- Cuando el ASG termina una instancia, la desregistra del LB (respetando el Connection Draining)
- El LB hace health checks a las instancias del ASG — si una instancia no está sana, el ASG la termina y crea una nueva

Políticas de Escalado

1. Dynamic Scaling (Escalado Dinámico) — Basado en métricas:

- **Target Tracking**: mantén una métrica en un valor objetivo. Ej: “mantén la CPU media del ASG al 70 %”. Simple de configurar
- **Step Scaling**: escala en pasos según alarmas de CloudWatch. Ej: “si CPU > 70 % añade 2 instancias; si CPU > 90 % añade 4 instancias”
- **Simple Scaling**: escala en un paso cuando se activa una alarma (más básico que Step)

2. Scheduled Scaling (Escalado Programado) — Por fecha/hora:

- Escala basándote en un horario conocido de antemano
- Ej: “cada viernes a las 18:00 sube la capacidad a 10 instancias (empieza el fin de semana)”
- Ej: “cada lunes a las 09:00 vuelve a la capacidad normal”

3. Predictive Scaling (Escalado Predictivo) — Machine Learning:

- AWS analiza el historial de carga y **predice** cuándo necesitarás más capacidad
- Escala de forma proactiva antes de que llegue el pico
- Ejemplo del curso: si cada primera semana de agosto vendes bañadores y siempre tienes pico, el escalado predictivo aprende ese patrón y pre-escala automáticamente antes de que llegue agosto

Métricas y CloudWatch Alarms

El escalado dinámico usa **alarmas de CloudWatch** que monitorizan métricas:

- **CPU Utilization**: porcentaje de CPU usado. La más común
- **RequestCountPerTarget**: número de peticiones por instancia
- **Average Network I/O**: tráfico de red promedio
- Métricas personalizadas que definas

Importante para el examen

Cooldown Period: después de una actividad de escalado, el ASG entra en un periodo de “cooldown” (por defecto 300 segundos) donde no escala de nuevo. Esto da tiempo a que las nuevas instancias se estabilicen y las métricas se normalicen antes de tomar otra decisión de escalado.

Consejo: usar AMIs preconfiguradas para que las instancias arranquen más rápido y el cooldown pueda ser más corto.

Ejemplo Práctico del Curso

Pasos para crear un ASG en la consola AWS:

1. Ir a EC2 → Auto Scaling Groups → Crear grupo
2. Asignar nombre: ej. `demo-asg`
3. Crear o seleccionar una Launch Template
4. Elegir subredes (seleccionar múltiples AZs para alta disponibilidad)
5. Asociar a un Load Balancer (target group existente)
6. Activar health checks del ELB (adicionales a los EC2)
7. Configurar capacidades: mínimo=1, deseado=2, máximo=4
8. Configurar políticas de escalado si se desea

9. Crear el grupo — el ASG lanzará automáticamente las instancias “deseadas”

Referencia Rápida para el Examen

Puertos importantes

- 22 →SSH / SFTP
- 21 →FTP
- 80 →HTTP
- 443 →HTTPS
- 3389 →RDP (Windows)
- 6081 →GENEVE (GWLb)

Tipos de Load Balancer

- HTTP/HTTPS/WebSocket →ALB (Capa 7)
- TCP/UDP/millones RPS →NLB (Capa 4)
- Firewall/inspección/GENEVE →GWLb (Capa 3)
- Multi-cert en un LB →SNI en ALB/NLB

EBS: cuándo usar cada tipo

- Boot + propósito general →GP2/GP3
- BD crítica alta IOPS →IO1/IO2
- Big Data / logs →ST1 (HDD)
- Acceso infrecuente, barato →SC1 (HDD)

- Millones de IOPS →**Instance Store**

Opciones de compra EC2

- Imprevisible/corto →On-Demand
- Estable 1-3 años →Reserved
- Máximo ahorro, interruptible →Spot
- Compliance/licencias →Dedicated Host
- Garantizar capacidad AZ →Capacity Reservation

IAM reglas rápidas

- Root →solo configuración inicial
- MFA →siempre activar
- Access Keys →como contraseñas, nunca en GitHub
- Deny explícito →siempre gana
- Roles →para servicios AWS, no personas

Diagnóstico de errores

- Timeout →Security Group bloqueando
- Connection refused →App no corre
- IP cambia al reiniciar →usar Elastic IP o DNS

Importante para el examen

Resumen de las analogías clave del curso:

- **Hotel Resort** →opciones de compra EC2 (On-Demand/Reserved/Spot/Dedicated/Savings Plans/Capacity Reservation)
- **Junior→Senior operador** →escalado vertical (scale up)
- **Añadir más operadores** →escalado horizontal (scale out)
- **Edificio NY + edificio SF** →Alta Disponibilidad entre AZs
- **Pendrive de red** →volumen EBS
- **Bolsa de instancias** →Target Group del ALB/NLB
- **MFA** →“si un hacker roba tu contraseña, sin el dispositivo físico no puede entrar”